



VPS/PEERTUBE/ – THE TOWER, PEERTUBE EDITION

DOC NO. RL-Z0-PEERTUBE-A REV. A RIPOSTE LABORATORIES

Pick one tower. Use this if your audience is big enough that egress is the bill (P2P only helps once several people watch at once); stay on Owncast if it isn't – Owncast is a single binary and this is a real stack.

CONTENTS

▶ What you trade vs Owncast	3
▶ Setup (VPS)	3
▶ Caddy	4
▶ Point the uplink at it – nothing in <code>playout/</code> changes	5
▶ Put it on the storefront	5
▶ Caveats worth repeating	6

A drop-in alternative to the Owncast tower (`../docker-compose.yml`). Same job – one RTMP stream in from home, HLS out to everyone – but PeerTube ships **peer-to-peer (WebRTC) segment sharing on by default**, so viewers feed each other and the VPS serves each segment far fewer times. On a linear channel, where everyone is watching the same thing at the same playhead, that swarm is about as good as P2P gets.

FILE	GOES WHERE	PURPOSE
<code>docker-compose.yml</code>	<code>/opt/peertube/</code>	PeerTube + Postgres + Redis: RTMP in (1935), HLS out (9000)
<code>.env.example</code>	<code>/opt/peertube/.env</code>	DB creds, secret, hostname, admin email

SEC.01

WHAT YOU TRADE VS OWNCAST

	OWNCAST (../)	PEERTUBE (HERE)
Moving parts	one Go binary	app + Postgres + Redis
RAM (live-only)	a few hundred MB	~2 GB realistic
P2P offload	added on the site side (<code>site/index.html</code>)	built into its own player
The catch	none, really	stateful deps to back up + upgrade; keep transcoding OFF

You may not need this at all. The Owncast tower + the P2P you already have in `site/index.html` (the `P2P_*` config) gets you swarm offload **without** Postgres/Redis. This directory is for when you'd rather the tower own the P2P and the player, chat, and accounts that come with PeerTube.

SEC.02

SETUP (VPS)

BASH

```
sudo mkdir -p /opt/peertube && cd /opt/peertube
# copy docker-compose.yml + .env.example here, then:
cp .env.example .env          # fill in a real DB password + PEERTUBE_SECRET
docker compose up -d
docker compose logs peertube | grep -A1 "Admin password"  # grab the one-time root password
```

Then, in the admin UI at `https://<your watch host>`:

1. Log in as `root` with that password; change it.

-
2. **Administration → Configuration → Live streaming → Enable live.**
 3. **Turn live transcoding OFF.** This is the money setting: with it off, PeerTube just remuxes the incoming RTMP into HLS (cheap, single-resolution, exactly like Owncast). With it on, ffmpeg re-encodes per resolution per stream and can eat several CPU cores – the opposite of why you’re here.
 4. Confirm P2P is on: it is by default (**Administration → Configuration → Advanced** keeps the tracker enabled – “if you disable the tracker, you disable P2P”).

SEC.03

CADDY

Same pattern as the Owncast tower – Caddy terminates TLS and proxies to the local port (9000 here instead of 8080). Put this alongside the block in `../Caddyfile`:

CADDY

```
watch.ch0.ripostelabs.xyz {  
    reverse_proxy 127.0.0.1:9000  
}
```

Then `sudo systemctl reload caddy`. (PeerTube serves the client + HLS itself, so unlike the Owncast block you don’t need the CORS header – the site embeds PeerTube’s own player in an iframe, not a cross-origin fetch.)

SEC.04 POINT THE UPLINK AT IT – NOTHING IN payout/ CHANGES

PeerTube's RTMP target has the **same shape** the uplink already pushes, `rtmp://<host>:1935/live/<stream-key>`, so the `uplink` and `.env` are untouched – only the **value** of `Z0_STREAM_KEY` changes to PeerTube's key:

1. In the UI, **Publish** → **Go live** → make it a **permanent/recurring** live (so the key is stable across sessions instead of one-shot).
2. The "Live settings" tab shows the RTMP URL and the stream key.
3. Put that key in `Z0_STREAM_KEY` and set `Z0_WATCH_DOMAIN` to this host. The uplink reconnects forever, same as before.

SEC.05 PUT IT ON THE STOREFRONT

PeerTube's player already does P2P, so on the site you embed it rather than running the custom `hls.js` player. In `site/index.html`'s `CONFIG`, set one line:

JS

```
PEERTUBE_EMBED: "https://watch.ch0.ripostelabs.xyz/videos/embed/<video-uuid>?p2p=1&muted=1",
```

The `<video-uuid>` is in the permanent live's URL. Setting it swaps the Live Feed section to PeerTube's `iframe` (with its built-in P2P + chat) and leaves the program grid, dossier, and the rest of the page exactly as they are. Leave it blank to keep the Owncast + P2P player.

SEC.06

CAVEATS WORTH REPEATING

- ▶ **Back up Postgres.** Owncast's config was a folder; here your instance lives in a database. `./data/postgres` is the important volume.
- ▶ **Domain + HTTPS are mandatory** – WebRTC needs a secure context and the P2P tracker runs over `wss://`. Caddy handles the certificate.
- ▶ **Live transcoding OFF** unless you have CPU to burn (or offload it to a PeerTube "remote runner"). This is the one setting that separates "as cheap as Owncast" from "melting the VPS."

SOURCE `vps/peertube/README.md` SHA `5f7dfd8c486a` BUILT `2026-08-11`